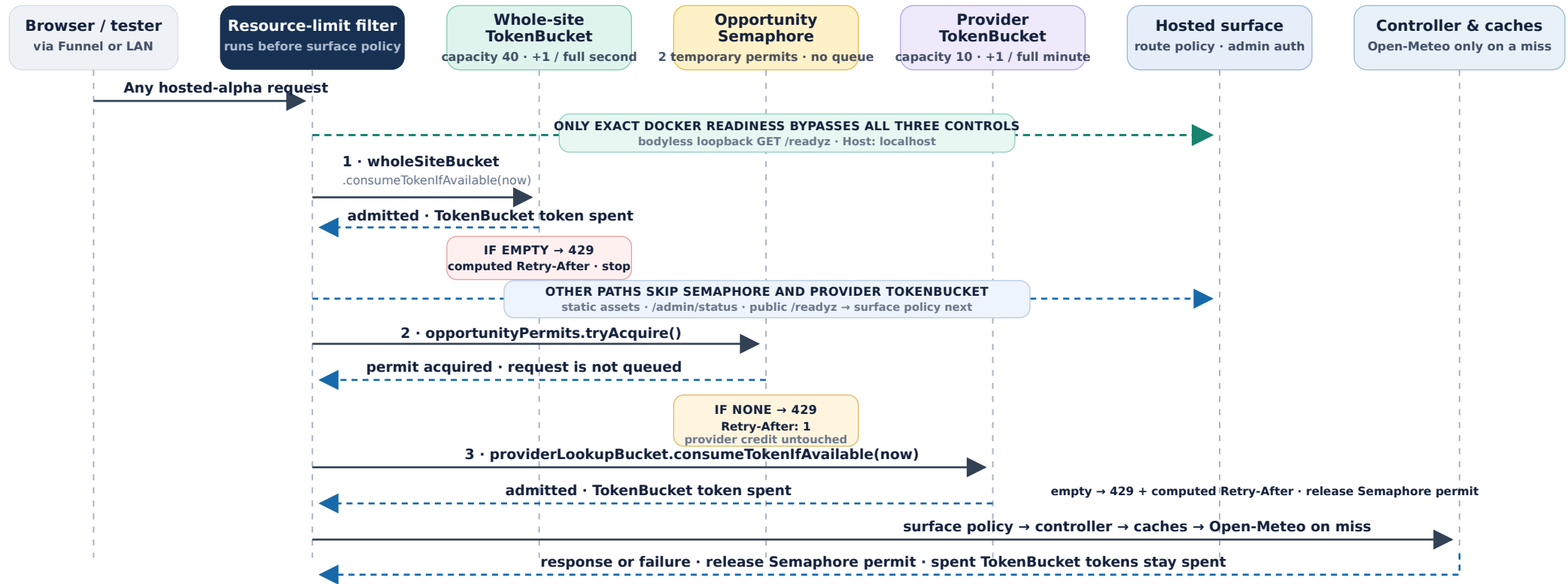


# Request admission: TokenBuckets and semaphore

The Spring filter spends shared admission credits before protected work—and rejects immediately when a bound is exhausted.

## Sequence for an admitted opportunity request

ONE PROCESS · SHARED BY ALL VISITORS



### TokenBucket mechanics

- A token is an admission credit—not the admin secret or an API key.
- Both buckets start full, are process-wide and synchronized.  
Refill accumulation is capped at each bucket's configured capacity.
- One credit returns for every complete refill interval.
- Partial elapsed time is preserved; a backward clock adds no credits.
- Retry-After rounds up to the next refill, with a 1-second minimum.
- A process restart fills both buckets again.

**Whole-site:** 40-credit burst + 1/sec refill = 60/min sustained  
Every request spends a credit except the exact Docker readiness probe.

### Semaphore ≠ TokenBucket

- Two semaphore permits bound in-flight opportunity searches.
- Acquisition is non-blocking: a third overlap gets 429 immediately.
- The rejected request already spent its whole-site credit.
- It does not spend a provider credit and it does not enter a queue.
- A permit is always returned when downstream handling finishes.

#### Permit lifetime

Includes validation, cache access, provider retries/backoff, response generation, and error handling.

### Provider-use bound

- Exact GET/HEAD /api/opportunities spends one provider credit.
- A cache hit still spends it; the credit admits a lookup, not a network call.
- One admitted lookup can make at most four Open-Meteo attempts:  
geocoding + one retry, then weather + one retry.
- /admin/status uses only whole-site capacity; its secret is later auth.

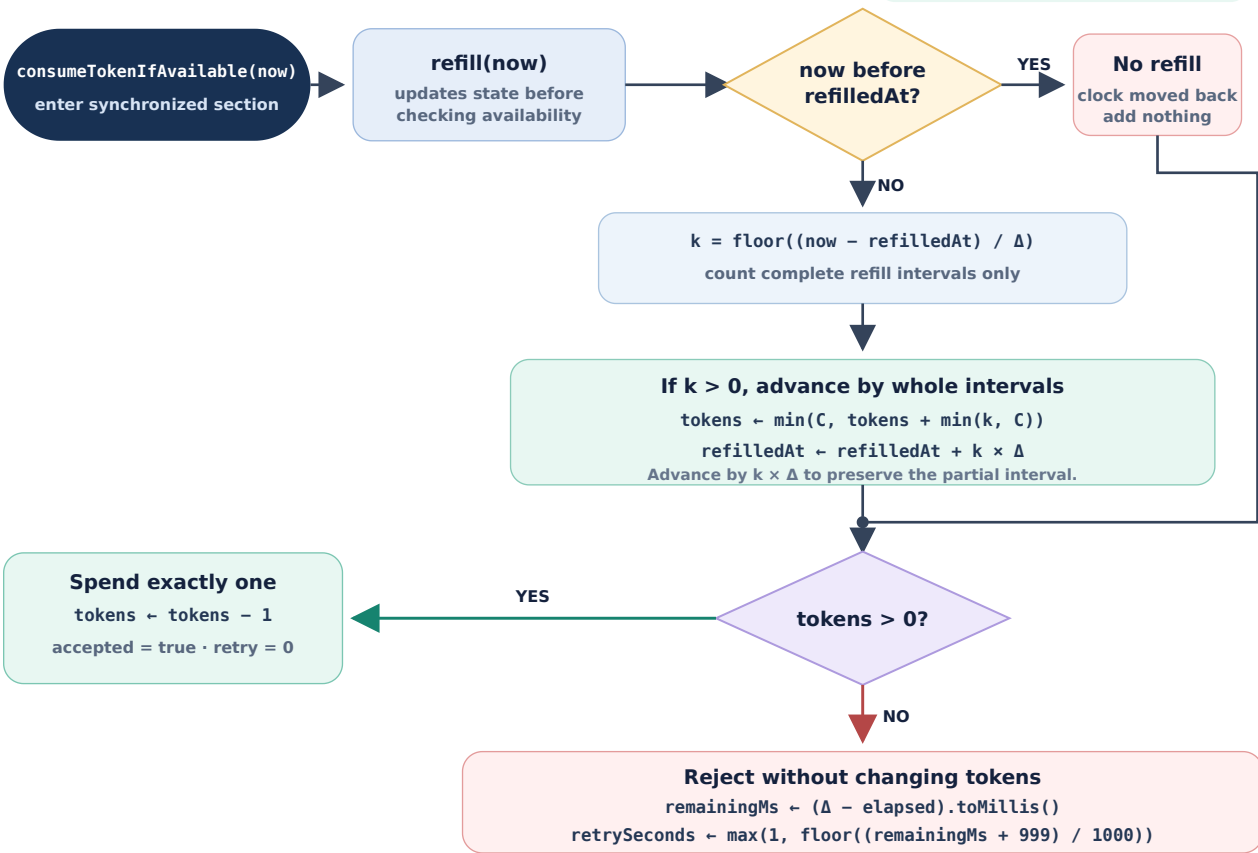
#### Uninterrupted 24-hour proof

10 initial + 1,440 refills = 1,450 admitted lookups  
1,450 × 4 = 5,800 worst-case provider attempts

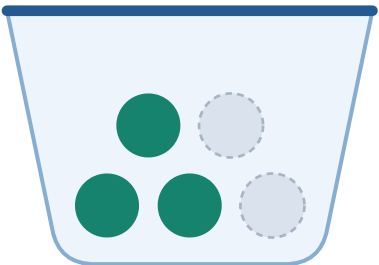
# Inside one TokenBucket

A synchronized call refills complete intervals, spends one admission credit, or returns a bounded retry hint.

## What synchronized consumeTokenIfAvailable(now) does LAZY REFILL · NO BACKGROUND TIMER



## +1 after each complete Δ State carried between calls



$0 \leq \text{tokens} \leq \text{capacity } C$   
illustrative slots; capacity is configurable

### Four fields

|                          |                                            |
|--------------------------|--------------------------------------------|
| capacity $C$             | maximum stored credits                     |
| refill interval $\Delta$ | time required for one credit               |
| tokens                   | current stored admission credits           |
| refilledAt               | anchor; advances even when capacity-capped |

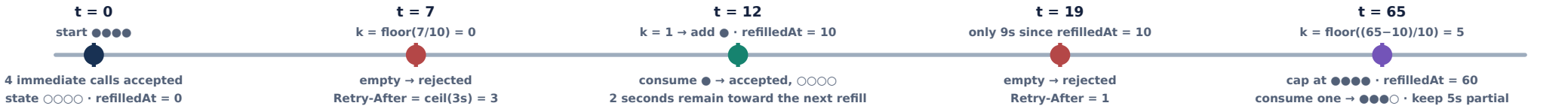
### Whole-site instance

$C = 40 \cdot \Delta = 1\text{s}$   
Shared by all non-Docker-probe requests in the process.

### Provider instance

$C = 10 \cdot \Delta = 60\text{s}$   
Independent bucket; same exact algorithm and process lifetime.

## Worked timeline · illustrative $C = 4, \Delta = 10$ seconds



### Clock and concurrency safeguards

A backward clock never mints credits; an empty bucket then uses zero elapsed time. Synchronization prevents simultaneous callers from spending the same credit. Downstream failure does not return a spent token; only timed refill or a process restart restores it.